Universidad de Alicante
Escuela Politécnica Superior

TECNOLOGÍA Y ARQUITECTURA ROBÓTICA (TAR)

Práctica 2

Turtlebot 3 en Entorno

Autor: Hernández Delgado, Jaime
Álvarez Bernabeu, Daniel
Mazón Muñoz, Manuel Jesús

Curso: 2025–2026

Repositorio GitHub:
https://github.com/nickhernd/TAR
Video de resolución de laberintos:
https://youtu.be/P-GyLycobWg

31 de marzo de 2026

Índice general

1. Parte 1: Primeros Pasos con el Turtlebot y Odometría **2**
1.1. Preguntas teóricas previas 2
1.2. Ejercicio 1: Figuras Geométricas 3
1.3. Ejercicio 2: Monitorización y Gráficas 4
1.4. Ejercicio 3: Desplazamiento Lineal Preciso 4
1.5. Ejercicio 4: Análisis de Precisión y Control 4

2. Parte 2: Resolución de un Laberinto **5**
2.1. Preguntas teóricas previas 5
2.2. Ejercicio 1: Resolución del Laberinto 1 — Algoritmo Wall Follower . . . 6
2.3. Ejercicio 2: Laberinto 2 y análisis de limitaciones del Wall Follower . . 8
2.4. Ejercicio 3: Algoritmo de Pledge para el Laberinto 2 9
2.5. Ejercicio 4: Comparativa Waffle vs. Burger en el Laberinto 1 11
2.6. Ejercicio 5: Laberinto propio (maze_3) 12

1. Parte 1: Primeros Pasos con el Turtlebot y Odometría

En esta sección se detalla la configuración inicial del entorno de simulación, la resolución de cuestiones teóricas sobre la cinemática del Turtlebot 3 y la implementación de nodos de control para trayectorias básicas y monitorización de odometría.

1.1 Preguntas teóricas previas

Preguntas 1 y 2: Topic y tipo de mensaje de movimiento

El topic en el cual se debe publicar la información para que el Turtlebot 3 se mueva es `/cmd_vel`. Podemos verificar tanto el topic como el tipo de mensaje ejecutando los siguientes comandos en una terminal paralela mientras el nodo de teleoperación (`teleop_keyboard`) está activo:

```
ros2 topic list
ros2 topic info /cmd_vel
```

El tipo de mensaje que se publica en este topic es `geometry_msgs/msg/Twist`. Este estándar de ROS contiene dos vectores tridimensionales: uno para expresar la velocidad lineal (`linear`) en los ejes X, Y, Z; y otro para la velocidad angular (`angular`) alrededor de dichos ejes.

Pregunta 3: Tipos de movimientos y ejes positivos

El Turtlebot 3, al poseer una configuración de tracción diferencial, está restringido a realizar movimientos en un plano 2D. Por tanto, emplea principalmente:

- **Movimiento lineal:** Desplazamiento hacia adelante o hacia atrás a lo largo de su eje longitudinal (`linear.x`).
- **Movimiento angular:** Rotación sobre su propio eje central (`angular.z`).

En ROS 2 se utiliza la regla de la mano derecha (estándar REP 103) para los sistemas de coordenadas. Esto implica que:

- El eje **X positivo** apunta hacia el frente del robot.
- El eje **Z positivo** apunta hacia arriba. Por tanto, una velocidad angular positiva produce un giro en sentido antihorario (**hacia la izquierda**).

En consecuencia, si le enviamos al robot un comando para que gire **-20º** (una velocidad angular negativa), el robot girará en sentido horario, es decir, **hacia la derecha**.

Pregunta 4: Giro sobre su eje y cinemática

Sí, el robot gira exactamente sobre su propio eje central vertical.

Esto se debe a su configuración cinemática de **tracción diferencial** (*differential drive*). Cuando enviamos un comando puramente rotacional a través de `/cmd_vel` (con velocidad lineal a 0), el controlador base del Turtlebot traduce este mensaje y manda a los motores de las ruedas comandos de velocidad de la **misma magnitud pero de signos opuestos**.

Por ejemplo, para girar a la izquierda, la rueda derecha recibe una velocidad positiva (avanza) y la rueda izquierda recibe una velocidad negativa (retrocede), provocando que el centro instantáneo de rotación (ICR) se sitúe exactamente en el centro del eje que une ambas ruedas.

Pregunta 5: Unidades de magnitud

De acuerdo con el estándar internacional adoptado por ROS (REP 103), las magnitudes enviadas en los mensajes de tipo `Twist` utilizan el Sistema Internacional de Unidades (SI):

- **Velocidad lineal:** Se mide en metros por segundo (**m/s**).
- **Velocidad angular:** Se mide en radianes por segundo (**rad/s**).

1.2 Ejercicio 1: Figuras Geométricas

Implementación del nodo `movimiento.py`

Para cumplir con el requerimiento de un único fichero ejecutable, se ha desarrollado una clase en Python orientada a objetos (`ControladorFiguras`) que integra un menú interactivo en la terminal. El control del movimiento se realiza en *lazo abierto*, calculando el tiempo de ejecución exacto necesario para cada desplazamiento y giro.

Fundamento Matemático: Al publicar mensajes del tipo `Twist` con velocidades constantes, el tiempo de activación (t) se calcula en función de la distancia (d), el ángulo a girar (θ) y las velocidades correspondientes (v lineal y ω angular):

$$t_{lineal} = \frac{d}{v} \quad ; \quad t_{giro} = \frac{\theta}{\omega} \tag{1.1}$$

Se establecieron los siguientes parámetros para las figuras:

- **Cuadrado:** El robot avanza $1\,m$ a $0{,}2\,m/s$ ($t = 5\,s$). Posteriormente, realiza un giro de $90°$ ($\pi/2\,rad$) a una velocidad de $0{,}5\,rad/s$ ($t \approx 3{,}14\,s$). Este patrón se repite 4 veces.

- **Triángulo equilátero:** El avance es idéntico al cuadrado. Sin embargo, para cerrar la figura, el robot debe rotar el ángulo exterior del polígono, equivalente a 120° ($2\pi/3\,rad$).
- **Infinito:** Esta figura consiste en la concatenación de dos circunferencias tangentes. Se logra enviando velocidad lineal y angular simultáneamente durante un tiempo correspondiente a $2\pi\,rad$. El primer círculo emplea una velocidad angular positiva ($\omega = 0{,}4\,rad/s$), y el segundo invierte el signo ($\omega = -0{,}4\,rad/s$).

Para garantizar la estabilidad en la simulación, se implementaron paradas de seguridad ($0{,}0\,m/s$) de 0,5 segundos entre las transiciones de movimiento lineal y angular.

Resultados en Simulación

Figura 1.1: Ejecución de las trayectorias en el entorno Gazebo.

1.3 Ejercicio 2: Monitorización y Gráficas

Análisis de Odometría con `dibuja_mov.py`

Se ha desarrollado un nodo que se suscribe al topic `/odom` para registrar la evolución temporal de la posición (x, y) del robot.

Figura 1.2: Trayectoria real registrada mediante odometría.

1.4 Ejercicio 3: Desplazamiento Lineal Preciso

Cálculo de distancia recorrida

Para asegurar un desplazamiento de exactamente 2 metros, se ha implementado un control basado en...

1.5 Ejercicio 4: Análisis de Precisión y Control

Problemas de lazo abierto y soluciones

- **Causas del error:** (Deriva, fricción, muestreo...)
- **Propuesta de mejora:** (Controlador Proporcional, etc.)

2. Parte 2: Resolución de un Laberinto

En esta parte se utiliza el Turtlebot 3 equipado con un sensor LiDAR para resolver laberintos de manera autónoma. Se emplea el paquete `maze_pkg`, que contiene dos nodos: `res_maze.py` (algoritmo *left-wall following*) y `pledge_maze.py` (Algoritmo de Pledge).

2.1 Preguntas teóricas previas

Pregunta 12: Topic y tipo de mensaje del LiDAR

El topic asociado al LiDAR del Turtlebot 3 es `/scan`. Lo verificamos con:

```
ros2 topic list
ros2 topic type /scan
```

El tipo de mensaje es `sensor_msgs/msg/LaserScan`, con los campos principales:

- `angle_min`, `angle_max`, `angle_increment`: rango y resolución angular.
- `range_min`, `range_max`: límites de distancia medibles.
- `ranges[]`: array con las distancias medidas en cada ángulo.
- `intensities[]`: intensidades de reflexión (vacío en simulación).

Pregunta 13: Rangos de distancia, ángulo de escaneo y origen de referencia

Los valores del LiDAR del Turtlebot 3 Waffle (sensor LDS-01) son:

Parámetro	**Waffle**	**Burger**
Rango mínimo	0.12 m	0.12 m
Rango máximo	3.5 m	3.5 m
Ángulo de escaneo	360°	360°
Resolución angular	1° (360 rayos)	1° (360 rayos)

Cuadro 2.1: Características del sensor LiDAR LDS-01 del Turtlebot 3

```
nickhernd@docker-desktop:/workspace$ ros2 topic echo /scan --
    once
header:
stamp:
sec: 13
nanosec: 113000000
frame_id: base_scan
angle_min: 0.0
angle_max: 6.28000020980835
angle_increment: 0.01749303564429283
time_increment: 0.0
scan_time: 0.0
range_min: 0.11999999731779099
range_max: 3.5
ranges:
- 2.4729459285736084
```

El origen de referencia es el propio sensor, montado en la parte superior del robot. El índice 0 del array `ranges[]` corresponde a la dirección de avance (*frente*), y los índices crecen en sentido antihorario: índice 90 $\approx$ izquierda, 180 $\approx$ atrás, 270 $\approx$ derecha.

Para el algoritmo se utilizan los siguientes sectores angulares:

- **Sector frontal**: índices correspondientes a $[-20, +20]$.
- **Sector izquierdo**: índices correspondientes a $[75, 105]$.

2.2 Ejercicio 1: Resolución del Laberinto 1 — Algoritmo Wall Follower

Se ha desarrollado el nodo `res_maze.py` dentro del paquete `maze_pkg`. El algoritmo implementado es *left-wall following* con una máquina de tres estados:

- **FIND_WALL**: el robot avanza en línea recta hasta detectar una pared al frente ($d_{\text{front}} < 0{,}45\,\text{m}$).
- **TURN_RIGHT**: el robot gira a la derecha sobre sí mismo hasta que el frente queda libre. Existe un contador mínimo `MIN_TURN_TICKS = 15` para evitar oscilaciones.
- **FORWARD**: avanza siguiendo la pared izquierda mediante un controlador proporcional. El error se define como $e = d_{\text{left}} - d_{\text{desired}}$; si la pared está lejos el robot gira a la izquierda y si está cerca gira a la derecha.

La función principal del nodo es el *callback* del LiDAR:

```python
def scan_callback(self, msg: LaserScan):
    front = min(safe(i) for i in range(-20, 21))
    left  = min(safe(i) for i in range(75, 106))

    if self.state == 'FIND_WALL':
```

```
        if front < FRONT_THRESH:
            self.state = 'TURN_RIGHT'
        else:
            cmd.linear.x = LINEAR_SPEED

    elif self.state == 'TURN_RIGHT':
        cmd.angular.z = -ANGULAR_SPEED
        self.turn_ticks += 1
        if self.turn_ticks >= MIN_TURN_TICKS and front >
           FRONT_THRESH + 0.15:
            self.state = 'FORWARD'

    else:  # FORWARD
        if front < FRONT_THRESH:
            self.state = 'TURN_RIGHT'
        else:
            error = left - WALL_DIST
            correction = max(-ANGULAR_SPEED, min(ANGULAR_SPEED,
               KP * error))
            cmd.linear.x  = LINEAR_SPEED
            cmd.angular.z = correction
```

Los parámetros principales del nodo son:

Parámetro	**Valor**	**Descripción**
`LINEAR_SPEED`	0.12 m/s	Velocidad lineal de avance
`ANGULAR_SPEED`	0.40 rad/s	Velocidad angular de giro
`WALL_DIST`	0.35 m	Distancia deseada a la pared izquierda
`FRONT_THRESH`	0.45 m	Umbral de detección de obstáculo al frente
`KP`	0.8	Ganancia del controlador proporcional
`MIN_TURN_TICKS`	15	Iteraciones mínimas en estado de giro

Cuadro 2.2: Parámetros del nodo `res_maze.py`

Para lanzar el ejercicio:

```
# Terminal 1 - Simulacion
export TURTLEBOT3_MODEL=waffle
ros2 launch maze_pkg maze_1.launch.py

# Terminal 2 - Solver
source /workspace/ros2_ws/install/setup.bash
ros2 run maze_pkg res_maze
```

El algoritmo Wall Follower resuelve correctamente el laberinto 1, ya que todas las paredes interiores están conectadas al perímetro exterior, condición suficiente para que el seguimiento de pared sea completo.

2.3 Ejercicio 2: Laberinto 2 y análisis de limitaciones del Wall Follower

Se ejecuta el mismo nodo `res_maze.py` con el segundo mapa, que presenta una topología diferente:

```
export TURTLEBOT3_MODEL=waffle
ros2 launch maze_pkg maze_2.launch.py
ros2 run maze_pkg res_maze
```

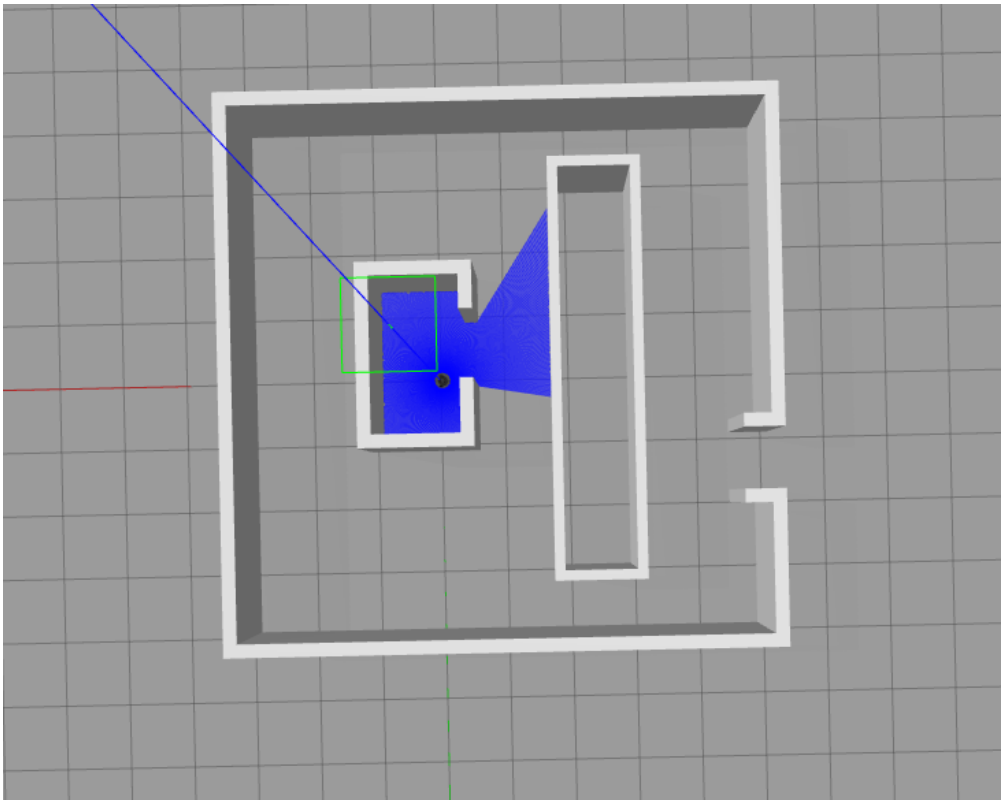

Figura 2.1: Turtlebot 3 Waffle atrapado en el laberinto 2: el robot orbita indefinidamente alrededor de la isla interior

Pregunta 14: Problemática observada en el laberinto 2

El laberinto 2 contiene **paredes interiores no conectadas al perímetro exterior** (islas). El algoritmo Wall Follower falla en este escenario porque:

1. El robot detecta la isla, la toma como pared de referencia y comienza a seguirla por la izquierda.
2. Al ser la isla un obstáculo cerrado (sin conexión a la salida), el robot completa el perímetro de la isla y vuelve al punto de inicio del seguimiento.
3. El proceso se repite indefinidamente: el robot queda atrapado en un bucle infinito rodeando la isla, sin llegar nunca a la salida.

Esta es la limitación fundamental del algoritmo: **solo garantiza la solución si el muro seguido está conectado topológicamente con la salida**.

Pregunta 15: ¿Es el robot capaz de resolver este laberinto? ¿Qué información necesita?

Con el algoritmo Wall Follower tal como está implementado, **el robot no es capaz de resolver el laberinto 2**. El robot queda atrapado orbitando alrededor de la isla interior sin encontrar la salida.

Para resolver este tipo de laberintos es necesario el **Algoritmo de Pledge** [1], una variante del Wall Follower que incorpora un *contador de giros netos*. La idea clave es: si el contador de giros regresa a cero mientras el camino está despejado, el robot abandona el muro y retoma la dirección original. Esto permite al robot *escapar* de las islas en lugar de orbitar alrededor de ellas. La implementación de este algoritmo se describe en el siguiente ejercicio.

2.4 Ejercicio 3: Algoritmo de Pledge para el Laberinto 2

Para resolver el laberinto 2 se ha implementado el nodo `pledge_maze.py` dentro del paquete `maze_pkg`. Este nodo implementa el **Algoritmo de Pledge**, descrito en el pseudocódigo siguiente:

Algoritmo de Pledge (seguimiento pared izquierda)

1. `contador` $\leftarrow 0$
2. **repetir:**
3. Avanzar en línea recta (*estado* NORTE) hasta encontrar un muro
4. Comenzar a seguir el muro izquierdo (*estado* MURO)
5. Incrementar/decrementar `contador` con cada giro
6. Si `contador` $= 0$ y el frente está libre $\Rightarrow$ volver al paso 3
7. **hasta:** salida encontrada

Cuadro 2.3: Pseudocódigo del Algoritmo de Pledge

Implementación en ROS 2

La clase `PledgeFinal` suscribe a `/scan` (LiDAR) y a `/odom` (odometría) para extraer el ángulo *yaw* actual mediante:

```
def odom_cb(self, msg):
    q = msg.pose.pose.orientation
    self.curr_yaw = math.atan2(
        2.0*(q.w*q.z + q.x*q.y),
        1.0 - 2.0*(q.y*q.y + q.z*q.z))

    if self.init_yaw is None:
```

```
        self.init_yaw = self.curr_yaw
        return

    # Contador de cuadrantes (-2, -1, 0, 1, 2)
    diff = self.normalize_angle(self.curr_yaw - self.init_yaw)
    current_q = int(round(diff / (math.pi / 2)))

    if current_q != self.last_q:
        step = current_q - self.last_q
        if step > 2:  step -= 4
        if step < -2: step += 4
        self.contador += step
        self.last_q = current_q
```

La máquina de estados principal, en el *callback* del LiDAR, tiene dos estados: `NORTE` (avanzar recto manteniendo la orientación inicial) y `MURO` (seguir la pared izquierda):

```
def scan_cb(self, msg):
    dist_frontal = self.get_dist_at_angle(msg, 0)
    dist_izq     = self.get_dist_at_angle(msg, 90)

    if self.state == 'NORTE':
        if dist_frontal < 0.45:
            self.state = 'MURO'  # Encontro muro -> empezar a seguirlo
        else:
            # Controlador P de rumbo: mantener orientacion inicial
            err = self.normalize_angle(self.init_yaw - self.curr_yaw)
            cmd.linear.x  = 0.15
            cmd.angular.z = 0.8 * err

    elif self.state == 'MURO':
        # Condicion de escape Pledge: contador=0 y orientacion inicial recuperada
        dist_norte = abs(self.normalize_angle(self.curr_yaw - self.init_yaw))
        if self.contador == 0 and dist_norte < 0.15 and dist_frontal > 0.6:
            self.state = 'NORTE'  # Escapar del muro
            return

        # Navegacion: seguir pared izquierda
        if dist_frontal < 0.35:
            cmd.angular.z = -0.5            # Girar derecha (resta al contador)
        elif dist_izq > 0.6:
            cmd.linear.x  = 0.1
            cmd.angular.z = 0.4             # Girar izquierda (suma al contador)
        else:
```

```
            cmd.linear.x  = 0.12
            cmd.angular.z = 1.3 * (dist_izq - 0.35)  # Control
                proporcional
```

La condición de escape es clave: cuando el `contador` vuelve a 0 **y** el robot ha recuperado su orientación original **y** el frente está libre, el robot abandona el muro. Esto garantiza que no se trata de un punto de paso al azar, sino de un estado en que el robot ha completado un número neto de giros igual a cero — condición necesaria para haber escapado de la isla.

Para lanzar el ejercicio:

```
# Terminal 1 - Simulacion
export TURTLEBOT3_MODEL=waffle
ros2 launch maze_pkg maze_2.launch.py

# Terminal 2 - Pledge solver
source /workspace/ros2_ws/install/setup.bash
ros2 run maze_pkg pledge_maze
```

El algoritmo de Pledge resuelve correctamente el laberinto 2. Cuando el robot rodea la isla por primera vez, el contador acumula giros; al completar el perímetro el contador vuelve a cero y el robot escapa en línea recta hacia la salida, en lugar de repetir el bucle.

El enlace al vídeo de la ejecución se puede encontrar en: `ENLACE_VIDEO_MAZE2_PLEDGE`

2.5 Ejercicio 4: Comparativa Waffle vs. Burger en el Laberinto 1

Se repite el Ejercicio 1 usando el modelo `Burger`:

```
export TURTLEBOT3_MODEL=burger
ros2 launch maze_pkg maze_1.launch.py
ros2 run maze_pkg res_maze
```

Pregunta 16: Diferencias observadas entre Waffle y Burger

Las diferencias observadas entre ambos modelos al resolver el laberinto son:

- **Dimensiones físicas**: el Burger (138 mm de diámetro) es más pequeño que el Waffle (281 mm), lo que le proporciona mayor holgura en pasillos estrechos y reduce el riesgo de colisiones laterales.
- **LiDAR**: ambos modelos utilizan el mismo sensor LDS-01 con cobertura de 360° y resolución de 1°, por lo que la percepción es equivalente y el mismo código funciona sin modificaciones.

- **Dinámica de giro**: el Burger tiene menor separación entre ruedas (*wheel separation*), lo que permite giros más cerrados pero también lo hace más sensible a acumulación de error odométrico.
- **Estabilidad**: el Waffle, más pesado (1.8 kg vs. 0.9 kg), es más estable a las velocidades empleadas. El Burger tiende a oscilar ligeramente más con los mismos parámetros del controlador proporcional.
- **Comportamiento en el laberinto**: ambos robots resuelven el laberinto 1 correctamente con los mismos parámetros, aunque el Burger lo hace con mayor margen de distancia a las paredes al ser más estrecho.

2.6 Ejercicio 5: Laberinto propio (maze_3)

Diseño del laberinto

Se ha diseñado un laberinto personalizado de **24×24 metros** (576 celdas de 1×1 m). Las características del diseño son:

- **Tamaño**: 24×24 m, 576 celdas, pasillos de 1 m de ancho y paredes de 0.15 m de grosor.
- **Una única salida**: hueco de 2 m en la pared norte, en $x \in [11, 13]$. No hay entrada; el robot se spawna directamente dentro en la posición (1,5, 1,5).
- **Sin islas simples**: la estructura base garantiza que todas las celdas son accesibles y las paredes forman un árbol generador.
- **Bucles añadidos**: se eliminaron manualmente 32 segmentos de pared adicionales para crear bucles internos, lo que hace que el Wall Follower simple quede atrapado y requiera el Algoritmo de Pledge para escapar.
- **Complejidad**: 288 segmentos de pared en total.

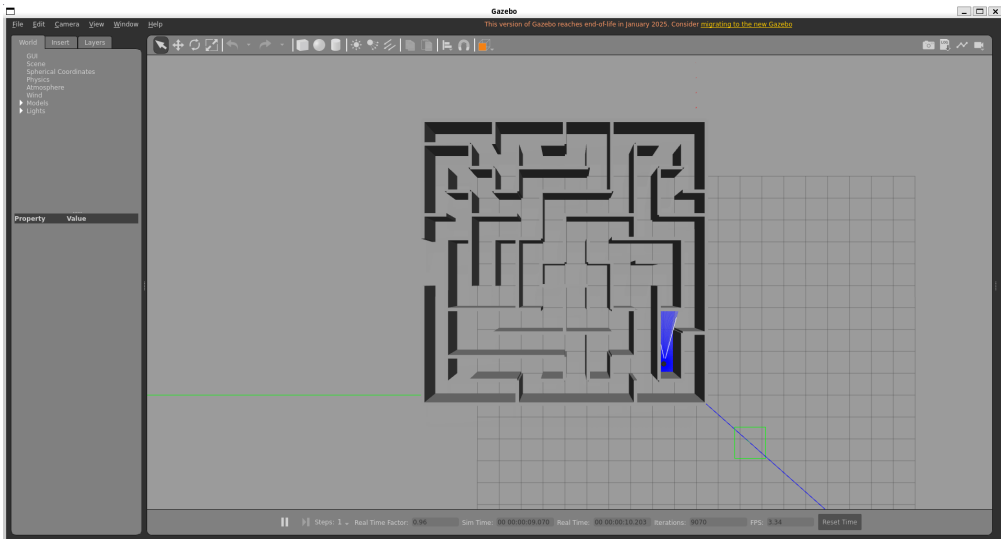

Figura 2.2: Vista cenital del laberinto propio (maze_3, 24×24 m) en Gazebo

Resolución con el Algoritmo de Pledge

Para lanzar el laberinto personalizado con el Algoritmo de Pledge:

```
# Terminal 1 - Simulacion
export TURTLEBOT3_MODEL=waffle
ros2 launch maze_pkg maze_3.launch.py

# Terminal 2 - Pledge solver
source /workspace/ros2_ws/install/setup.bash
ros2 run maze_pkg pledge_maze
```

El Algoritmo de Pledge resuelve satisfactoriamente el laberinto 3. Los bucles introducidos en el diseño atrapan al Wall Follower simple, pero el contador de cuadrantes del Pledge permite al robot escapar de cada bucle en cuanto los giros netos vuelven a cero y la dirección norte está libre. Dado el tamaño del laberinto (24 m), el robot recorre un camino significativamente más largo antes de encontrar la salida, lo que pone de manifiesto la robustez del algoritmo frente a laberintos de gran escala.

Bibliografía

[1] R. Klein and T. Kamphans, *Pledge's Algorithm — How to Escape from a Dark Maze*, pp. 69–75, Springer Berlin Heidelberg, 2011. https://doi.org/10.1007/978-3-642-15328-0_8